

1. 구현한 기능 목록

1) Firebase 회원 데이터 연동

- **Firebase Realtime Database**를 이용하여 사용자 정보를 저장 및 관리하였다.
- 로그인 시 사용자 **Key**를 기반으로 **Firebase**의 사용자 데이터를 불러온다.
- 플레이어의 코인, 인벤토리 정보를 관리한다.

2) 아이템 상점 기능

- 게임 내에서 구매 가능한 아이템을 판매하는 상점을 구현하였다.
- 아이템 구매 시 사용자의 코인이 차감되고, 구매한 아이템이 인벤토리에 추가된다.
- 구매한 데이터는 **Firebase Database**에 저장하여 재접속 시에도 유지된다.

3) 인벤토리 기능

- 사용자가 보유한 아이템 목록을 **Dictionary** 형태로 관리하였다.
- 아이템 이름을 **Key**, 보유 개수를 **Value**로 저장하였다.
- **Firebase** 저장을 위해 **JSON** 형태로 변환하여 관리하였다.

4) 거래소 기능

- 사용자가 보유한 아이템을 판매 등록할 수 있도록 구현하였다.
- 판매 등록 시:
 - 사용자의 인벤토리에서 아이템 개수가 감소
 - **Firebase Market** 데이터에 판매 정보 저장
- 다른 사용자가 구매하면:
 - 구매자의 코인 감소
 - 구매자의 인벤토리에 아이템 추가
 - 판매자의 코인 증가
 - 거래 완료 후 **Market** 데이터 삭제

5) 로그아웃 기능

- 로그아웃 시 **PlayerPrefs**에 저장된 로그인 정보를 삭제한다.
- 현재 로그인 중인 사용자 데이터를 초기화한다.

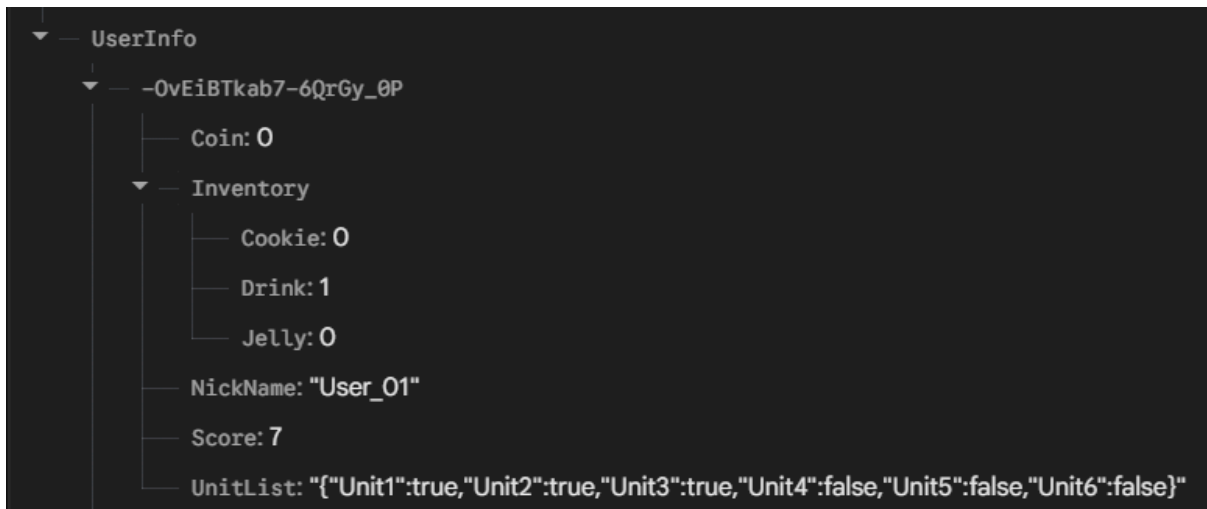
2. 제작한 아이템 설명

아이템 이름	사용 메세지
Drink	시원한 음료를 마셨습니다.
Cookie	바삭한 쿠키는 맛있습니다!
Jelly	젤리를 먹었습니다!

3. 아이템 가격표

아이템 이름	구매 가격	판매 가격
Drink	200	100
Cookie	300	200
Jelly	100	50

4. Firebase 데이터 구조 설명



UserInfo

- 사용자 개인 정보를 저장한다.
- Coin, Inventory 등의 게임 데이터를 관리한다.



Market

- 판매 등록된 아이템 정보를 저장한다.
- Firebase Push Key를 이용하여 각각의 거래 데이터를 구분한다.

5. PlayerPrefs에 저장한 값 설명

PlayerPrefs는 로그인 상태 유지를 위해 사용하였다.

Key	저장 값	용도
UserKey	Firebase 사용자 Key	현재 로그인한 사용자 식별
UserNickName	사용자 닉네임	거래소 판매자 이름 표시

6. Inventory JSON 처리 방식

Firebase Realtime Database에서는 Dictionary 데이터를 JSON 형태의 트리 구조로 저장할 수 있기 때문에, Inventory를 별도의 JSON 문자열로 변환하지 않고 아이템별 데이터로 나누어 저장하였다.

7. 실행 중 발생한 문제와 해결 방법

문제 1. 인벤토리 또는 코인의 동기화 문제

원인 : InventoryManager ShopManager 가 각각 인벤토리 정보를 따로 들고 있어서 다른 값을 보고 있었다.

UserData를 적극 활용하여 하나의 데이터를 공유하도록 수정 하였다.